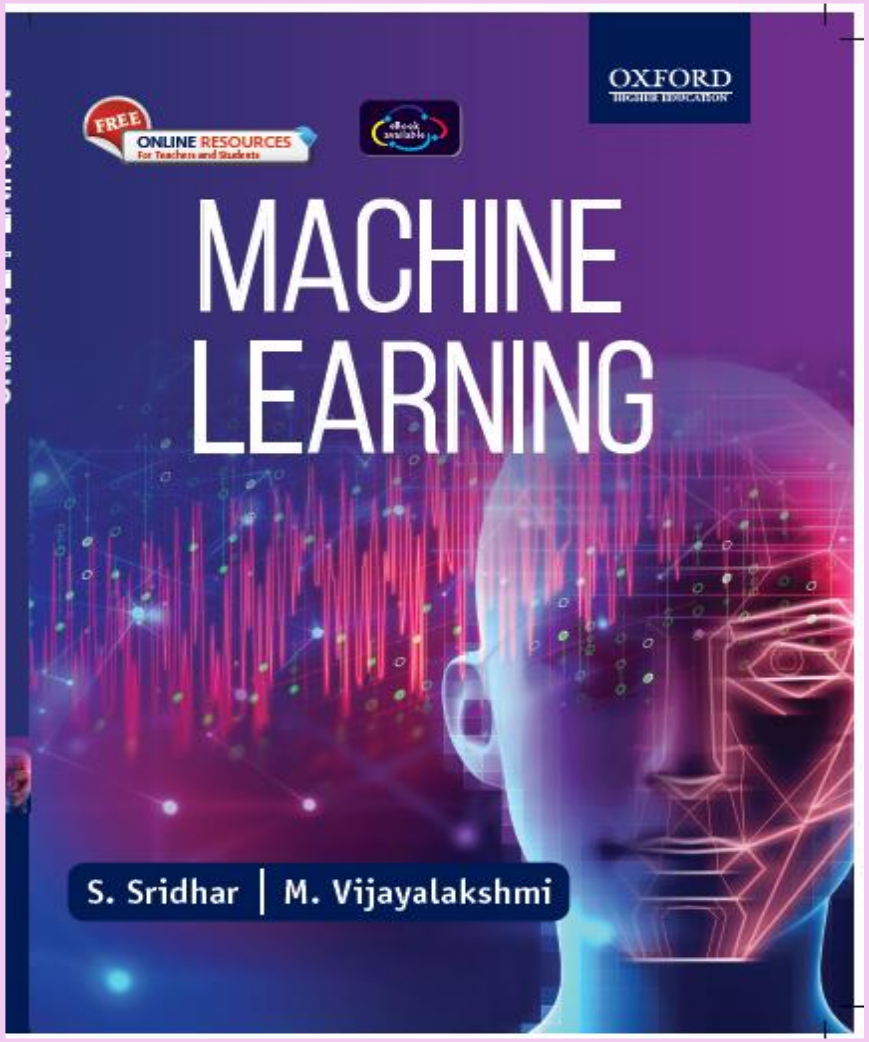


OXFORD
UNIVERSITY PRESS

Machine Learning

S. Sridhar and M. Vijayalakshmi

© Oxford University Press 2021. All rights reserved

Chapter 7

Rule-based Learning

© Oxford University Press 2021. All rights reserved

Rule–based Learning

Rule-based Learning represents knowledge in the form of IF-THEN rules that proves useful in artificial intelligence (AI) systems. This type of learning is most suitable for analyzing data containing a mixture of numerical and qualitative attributes. Rule mining is useful and easy to interpret by human users. The aim of this learning is to discover interesting relations between variables and patterns in large datasets. Several algorithms have proven useful which automatically induce rules from data to build more accurate AI systems. There are two ways by which we can generate classification rules:

1. Direct method which extracts rules directly from data. Some examples of direct classification models are OneR algorithm and Sequential Covering algorithms.
2. Indirect method which extracts rules from other classification models. Decision Trees and Genetic algorithms belong to the category of indirect classification models.

© Oxford University Press 2021. All rights reserved

Characteristics of Rule-based classification

ASSOCIATION RULES

Association rules are simple IF-THEN statements that show the probability of relationships between data items in a large dataset. An association rule has two parts, called an antecedent (-IF- part) and a consequent (-THEN- part). An antecedent/precondition is the test condition that consists of one or more attribute-value pairs logically ANDed (conjunction of the attribute-value pairs) and evaluated as true or false for each instance in the dataset. A consequent is the conclusion or the class prediction.

Association rules are expressed as:

IF condition THEN conclusion

RULE EVALUATION METRICS

Rules are generally assessed by two parameters, called Coverage and Accuracy.

1. Coverage = Number of instances that satisfy the antecedent of a rule/total number of instances in the dataset.
2. Accuracy = Number of instances that satisfy both the antecedent and consequent of a rule/ number of instances that satisfy the antecedent of a rule.

© Oxford University Press 2021. All rights reserved

Characteristics of Rule-based classification

RULE EXECUTION STRATEGY

Rule-based classification models generate more than one rule which may be conflicting, and an order has to be imposed in executing the rules. There are two ways of executing a rule set:

1. An unordered set of rules where rules may overlap that leads to different conclusions for the same test instance.
2. An ordered set of rules which is like a 'decision list' or 'priority list' and an order is imposed to classify a test instance.

If there exists more than one rule to classify a test instance, conflict resolution strategies need to be incorporated. On the other hand, if there exists no rule to classify a test instance, then a default class is used.

Conflicts among the different rules that satisfy the same test instance can be resolved by imposing ordering among the rules. One approach is to do size ordering, which involves assigning priority based on the antecedent size. Another approach is ordering the rules based on the most frequent target class appearing first, then moving on to the next most frequent class and so on, or by the decreasing order of misclassification cost per class. But, the most popular strategy is to order the rules in the form of a 'decision list' or 'priority list' based on the rule quality measures such as coverage and accuracy or by expert knowledge.

© Oxford University Press 2021. All rights reserved

Sequential Covering Algorithm

Algorithm 7.1: General Sequential Learning

Input:

T: Given training dataset

Output: Rule-Set (RS) which is a set of IF-THEN rules

Procedure:

Step 1: Initialize Rule-Set (RS) = { } and Updated-Dataset (U) = T.

Step 2: For each target class C_i, repeat the following steps until U contains instances for class, C_i:

(a) Learn one rule R_i that correctly classifies some instances in U belonging to target class, C_i.

(b) Eliminate instances covered by rule R_i from U.

(c) Add rule R_i to Rule-Set (RS).

© Oxford University Press 2021. All rights reserved

PRISM

Algorithm 7.2: PRISM

Input:

T: Given training dataset

α_i: Attribute-value pair

C_i: Specific target class

Output: Rule-Set (RS) which is a set of IF-THEN rules

Procedure:

Step 1: Initialize Rule-Set (RS) = $\{R_i = \{\} \rightarrow C_i\}$ and Updated-Dataset (U) = T.

Step 2: For each target class C_i, repeat the following steps until U contains instances for class, C_i:

(a) Learn rule R_i until perfect (i.e., accuracy = 1).

Rule Learning process:

- For each attribute-value pair (α_i), compute the accuracy of the attribute-value pair (α_i):
  - **P (C_i | α_i)** is the Probability of occurrence of C_i given α_i
- Choose the attribute-value pair (α_i) of maximum accuracy over U, and in case of a tie, choose the pair that covers more number of instances of U.
- Create a subset $S\alpha_i$ that contains instances with α_i and repeat the above two steps for the subset until it contains only instances for target class C_i. The rule is formed by a conjunction of all the attribute-value pairs used in creating the subset.
- Update rule R_i = {Attribute* – Value* (α_i*) $\rightarrow C_i$} by appending this learned Attribute* – Value* pair α_i* to the antecedent part.
- Eliminate instances covered by rule R_i from U.

Step 3: Add this rule R_i to Rule-Set (RS).

© Oxford University Press 2021. All rights reserved

First Order Rule Learning

Learning rules in first order form are much more expressive than propositional logic when representing knowledge. The first order rule learners are capable of expressing relations between attributes which cannot be expressed by propositional rule learners. Moreover, first order rule learners can also acquire recursive rules.

First order logic, also called as predicate logic, is a powerful tool that expresses the relationship between objects. The syntax of rules written in first order logic consists of basic elements called as terms.

Each term can be expressed as:

Constant: Sam, Mathan, Ayden, etc.

Variables: x, y, z, etc.

Predicates or Relationships: Father, Son, Female, GrandFather, etc.

Function: Weight (Sam), age (Mathan), etc.

Connectives: $\wedge$, $\vee$, $\neg$, $\rightarrow$ (or $\leftarrow$)

Equality: ==

Quantifier: $\forall$, $\exists$

The basic sentences in the first order logic are called as atomic sentences.

The syntax of atomic sentence is represented as:

Predicate (term 1, term 2, .. term n)

© Oxford University Press 2021. All rights reserved

First Order Logic Terminology

- Every expression is composed of terms.
- A ***term*** can be a constant, or variable, or any function applied to a term.

  Example: Constants: Sam, Mathan

  Variables: x, y, z

  Predicates: Father, Son, Male

  Function: Weight (Sam)

  Connectives: $\wedge$, $\vee$, $\neg$, $\rightarrow$ (or $\leftarrow$)

  Quantifiers: $\forall$, $\exists$
- A ***literal*** is a predicate or a negated predicate. Predicates express the relations between terms.

- A ***Clause*** is a disjunction of literals whose variables are universally quantified.

  Example: $\forall x$: Female $(x) \vee$ Male (x)

© Oxford University Press 2021. All rights reserved

First Order Logic Terminology

- A ***Horn clause*** is a clause that has one or more preconditions or literals in disjunctive form and one single consequent. It is expressed in the form:

$$H \leftarrow (L_1 \wedge L_2 \wedge \ldots \wedge L_n)$$

where, H, L_1, L_2, L_n are positive literals. The term in the LHS 'H' is called the *head, post-conditions* or *consequent* of the Horn clause. The conjunction of the literal $L_1 \wedge L_2 \wedge \ldots \wedge L_n$ in the RHS is called the *body,* precondition or *antecedents* of the Horn clause.

Using the law, $(A \leftarrow B) \Rightarrow (A \vee \neg B)$, the horn clause $H \leftarrow (L_1 \wedge L_2 \wedge \ldots \wedge L_n)$ can be re-written as:

$$H \vee \neg L_1 \vee \neg L_2 \vee \ldots \vee \neg L_n$$

- A ***substitution*** is any function that replaces variables by terms.

Example: $\{x/\text{Sam}, y/z\}$, here x is replaced by the term Sam and y is replaced with the variable z.

© Oxford University Press 2021. All rights reserved

FOIL (First Order Inductive Learner Algorithm)

Algorithm 7.3: FOIL

```
FOIL (consequent-predicate, Background knowledge B, Training examples)
P = Positive examples
N = Negative examples
RuleSet = { }
      while P is not empty do //outer loop
{
      //learn a new rule
      newRulenewR = consequent-predicate← { }
      N1 = N
      while N1 is not empty do //inner loop
      {
      //add a new literal to R
            Generate candidate literals for newR based on predicates
            Choose the literal L that gives maximum Foil-Gain by adding the literal
            to newR
            add L to preconditions of newR
            N1 = subset of N1 that satisfies newR preconditions
      }
      RuleSet = RuleSet + newR
         P = P − {positive examples covered by newR}
      }
      return RuleSet
```

© Oxford University Press 2021. All rights reserved

FOIL's Information Gain (FOIL–Gain)

The Foil-Gain function is used to select the best literal. Foil-Gain is calculated using the below formula when a rule is greedily grown,

$$\textbf{FOIL–Gain}\ (R_0, R_1) = p_1 \times \left[\log_2 \frac{p_1}{p_1 + n_1} - \log_2 \frac{p_0}{p_0 + n_0} \right] \quad (7.1)$$

where,

R_0 – rule before adding a literal (consequent-predicate ← { })

R_1 – rule after adding a literal as a conjunct to R_0(consequent-predicate ← {L})

p_0 – number of positive examples/instances covered by R_0

p_1 – number of positive instances covered by R_1

n_0 – number of negative examples/instances covered by R_0

n_1 – number of negative instances covered by R_1

© Oxford University Press 2021. All rights reserved

Generating Candidate Specializations

If the current rule is $P(x_1, x_2, \ldots, x_k) \leftarrow L_1 \ldots L_n$ where $L_1 \ldots L_n$ are the preconditions and $P(x_1, x_2, \ldots, x_k)$ is the head of the rule, FOIL generates candidate literals by considering new literals L_{n+1} that fit one of the following forms:

1. $Q(v_1, \ldots, v_r)$ where Q is any predicate in the given predicates and v_i represents new or already present variables. At least one v_i in the generated literal must already be present in the rule.
2. Equal (x_i, x_j) where x_i and x_j are already present in the rule
3. Negation of either of the above

For example, if there is a predicate $Q(x_1, x_2)$, FOIL generates Candidate-literals as $Q(x_1, x_2)$, $Q(x_2, x_1)$, $Q(x_1, x_3)$, $Q(x_3, x_1)$, $Q(x_3, x_2)$, $Q(x_2, x_3)$, Equal (x_1, x_2) and the negation of each.

© Oxford University Press 2021. All rights reserved

Induction As Inverted Deduction

Another approach for Inductive inference is, in fact, the inverse operation of deduction. Inductive algorithms carry out automated deduction of finding hypothesis H for the target function with the help of evidence E (specific instances of the training data) and background knowledge B. Here, learning is a combination of deductive inference and formal logic that finds a general conclusion that is most likely to fit the specific instances and is used when making predictions, creating generalizations, and analyzing cause and effect.

Similarly, inductive inference derives a general conclusion C from evidence (specific instances) E.

$$C \models E$$

This expression is read as, E follows deductively from C.

As with ILP, the above elements are expressed into evidence (specific instances) (E), background knowledge (B), and hypothesis (H). These have the relationship as:

$$B \wedge H \models E$$

B, H and E are each logic programs. 'E' usually consists of positive examples (P) and negative examples (N). Given the formulas $B \wedge H \models E$, deriving E from $B \wedge H$ is deduction, and deriving H from B and E is induction.

© Oxford University Press 2021. All rights reserved

Inverting Resolution

Resolution is a general method for automated deduction introduced by Robinson in 1965. It is a complete method to perform deductive inference using the first order predicate calculus and could be affected by the single Resolution rule of inference.

RESOLUTION OPERATOR (PROPOSITIONAL FORM)

Resolution rule in propositional form can be shown as follows:

Given L is a propositional literal and C_1 and C_2 are propositional clauses, the resolution rule can be written as:

$$C_1: A \vee L \text{ and } C_2: \neg L \vee B \qquad \text{Resolvent } C = A \vee B$$

© Oxford University Press 2021. All rights reserved

Inverting Resolution

INVERSE RESOLUTION OPERATOR (PROPOSITIONAL FORM)

From resolution operator, inverse resolution can be performed which performs the induction inference. It means when given a clause and resolvent, we can infer the other clause.

Consider, the resolvent C: $A \vee B$ and the initial clause C_1: $A \vee D$, C_2 can be derived by inverse resolution operator as: C_2: $B \vee \neg D$. This C_2 can also be derived as: $B \vee \neg D \vee A$. Hence, inverse resolution is not deterministic; there can be more than one possible clauses for C_2.

Given initial clause C_1 and C, find a literal L that occurs in C_1 but not in clause C. Then, from C and C_1, form the second clause C_2 by including the following literals except for L and $\neg L$.

$$C_2 = (C - (C_1 - \{L\})) \cup \{\neg L\}$$

© Oxford University Press 2021. All rights reserved

Inverting Resolution

FIRST ORDER RESOLUTION

This resolution rule can be easily applied to first order rules. Here, it is processed by unifying substitutions. Substitution means mapping of variables to terms.

Consider an example literal Father (y, z) and the substitution θ = {x/Roselin, y/Ayden, z/Mathan}, then $L\theta$ = Father (Ayden, Mathan).

We can say θ is an unifying substitution for two literals L_1 and L_2 if $L_1\theta = L_2\theta$.

If we are given two literals L_1 and L_2 as L_1: Father (x, y), L_2: Father (Ayden, z) and θ: {x : Sam, z/y}, then by unifying substitution $L_1\theta = L_2\theta$ = Father (Sam, y).

Thus, resolution rule for first order expression is generalized as:

Find a literal L_1 from clause C_1, literal L_2 from clause C_2, and substitution θ such that $L_1\theta = \neg L_2\theta$. Then form the resolvent C by including all literals from $C_1\theta$ and $C_2\theta$, except for $L_1\theta$ and $\neg L_2\theta$.

More precisely, the set of literals occurring in the conclusion C are:

$$C = (C_1 - \{L_1\})\theta \cup (C_2 - \{L_2\})\theta.$$

© Oxford University Press 2021. All rights reserved

Inverting Resolution

INVERTING FIRST ORDER RESOLUTION

We can derive the inverse resolution from the resolution rule for first order expression.

The unifying substitution θ can be uniquely factored into θ_1 and θ_2, where $\theta = \theta_1\ \theta_2$. θ_1 contains all substitutions for variables in clause C_1 and θ_2 contains all substitutions for variables in clause C_2. Hence, we can write:

$$C_2 = (C - (C_1 - \{L_1\})\theta_1)\theta_2^{-1} \cup \{\neg L_1\theta_1\theta_2^{-1}\}$$

© Oxford University Press 2021. All rights reserved

Example 7.2:

Example 7.2: Learn the rules using inverse resolution rules for the target predicate GrandChild (x, y). Given, Training data: GrandChild (Roselin, Ayden) and Background Knowledge B: {Father (Ayden, Mathan), Mother (Mathan, Roselin)}

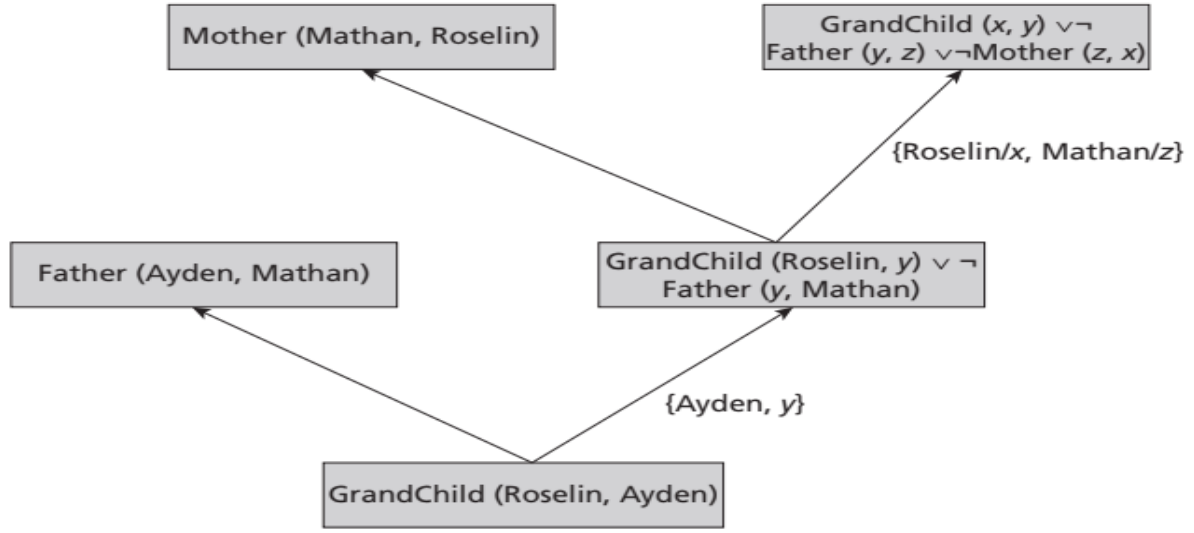


Figure 7.4: Steps of Inverse Resolution

Thus, the general rule is inferred as:

$$\textbf{GrandChild}\ (x, y) \leftarrow \textbf{Father}\ (y, z) \wedge \textbf{Mother}\ (z, x)$$

© Oxford University Press 2021. All rights reserved

Analytical Learning or Explanation Based Learning (EBL)

Analytical learning uses a combination of prior knowledge of the domain and deductive reasoning to learn a concept. Analytical reasoning is based on logical reasoning rather than statistical reasoning to generalize the concept. The logic based analytical reasoning or rather EBL is to use priori knowledge (called domain knowledge) to analyze or explain the training samples to check whether it satisfies the target concept. Then, this explanation is used to generalize the concept.

ADVANTAGES OF ANALYTICAL LEARNING

1. Inductive learning is more data-intensive and data-driven, whereas EBL is more knowledge-intensive and knowledge-driven.
2. EBL uses the priori knowledge to reduce the complexity of the hypothesis space.
3. Generalization is improved in case of analytical learning.

© Oxford University Press 2021. All rights reserved

Perfect Domain Theories

The domain theory includes background knowledge that can be used to explain or analyze the training examples which later can be used to generalize. Thus, the role of domain theories is a must for analytical reasoning. The domain theory is called correct if and only if its assertions are true with respect to the real world. The domain theory is called complete if it covers all the positive examples of the training dataset.

© Oxford University Press 2021. All rights reserved

Architecture of Explanation-based Learning

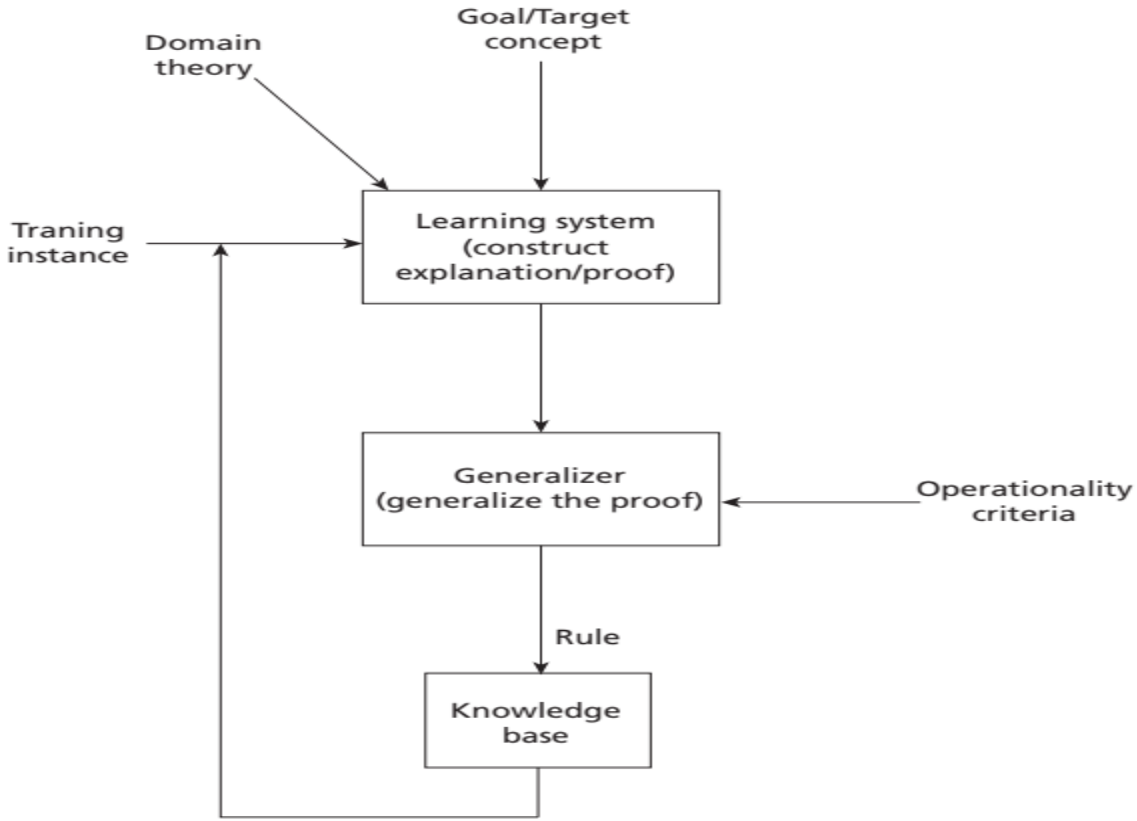


Figure 7.5: Architecture of Explanation-based Learning

© Oxford University Press 2021. All rights reserved

Example 7.3:

LEARN WHEN A FRUIT IS AN ORANGE USING PERFECT DOMAIN THEORIES

Let us assume the following domain theory about oranges that includes the following rules:

1. Edible (X) ^ Juicy (X) → Orange (X)
2. Fleshy (Z, T) ^ Contains_Water (T) ^ Fruit (T) → Juicy (Z)
3. Raw (Y) ^ Fleshy (Y, inside) →Edible (Y)
4. Nutritious (S) → Raw (S)
5. made_of (S, enzymes) → Raw (S)

The training example is an instance of the goal concept. That is, we are given:

Orange (fruit1)
Nutritious (fruit1)
Fleshy (fruit1, inside)
Eats (Ayden, fruit1)
Fleshy (fruit1, skin)
Fruit (skin)
Contains_Water (skin)
Color (fruit1, orange)

© Oxford University Press 2021. All rights reserved

Example 7.3:

STEP 1-PROVE THAT ORANGE IS A FRUIT

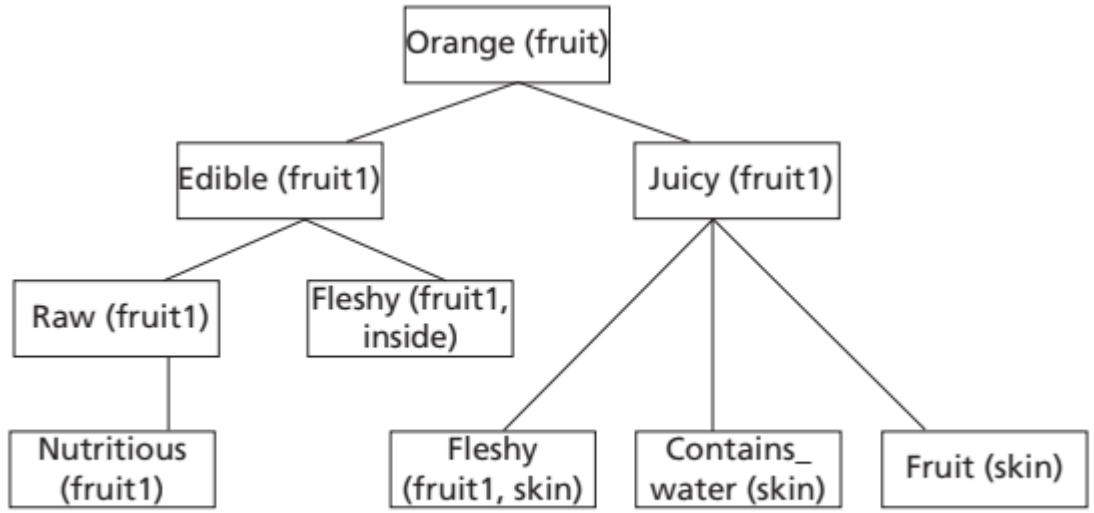


Figure 7.6: Proof for the Instance 'Orange is a Fruit'

© Oxford University Press 2021. All rights reserved

Example 7.3:

STEP 2- GENERALIZE THE PROOF

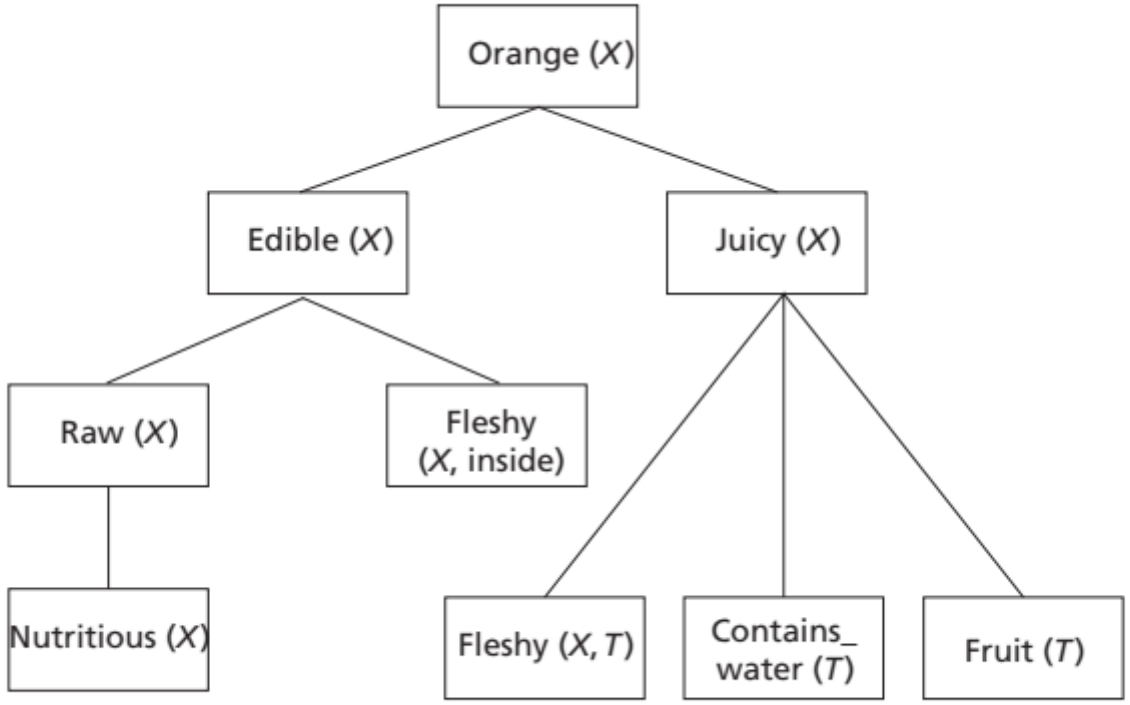


Figure 7.7: Generalizing the Proof

© Oxford University Press 2021. All rights reserved

Active Learning

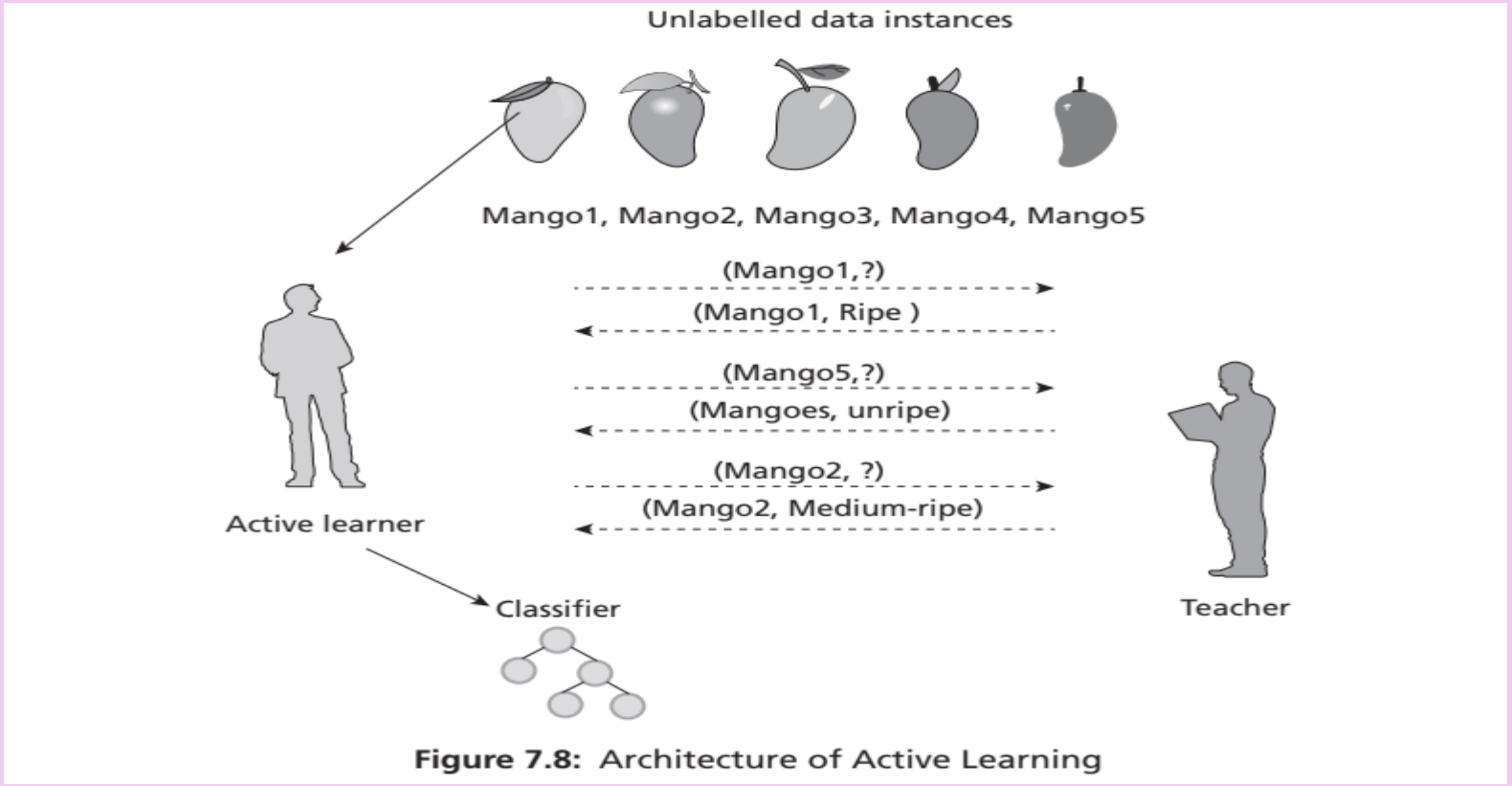


Figure 7.8: Architecture of Active Learning

© Oxford University Press 2021. All rights reserved

Active Learning Mechanisms

Membership Query Synthesis In this method, an instance is generated or constructed by the learner and this created instance is sent to the teacher/oracle for labelling.

Stream-based Selective Sampling In this method, an unlabelled instance is selected at a time by the learner who decides whether to query for labelling by the teacher/oracle or to discard it. The learner's decision is based on some informative measure. Each instance is considered separately when querying for labelling or for rejection.

Pool-based Sampling In this method, many instances are chosen from a pool of unlabelled data instances for labelling. The chosen instances are considered as the most informative instances based on some informative measure. This sampling is used in many real-world problems such as text classification, information extraction, video classification, image classification and retrieval, speech recognition, etc.

© Oxford University Press 2021. All rights reserved

Query Strategies/Selection Strategies

Random Sampling

It is a common baseline strategy where 'k' unlabelled samples are selected at random from the pool.

Uncertain Sampling

This is a traditional method where 'k' samples, about which the learner is least certain of how to label, are selected from the pool.

Least Confidence (LC) In this strategy, the active learner selects 'k' samples in the pool which has the least confident value about its label. When the prediction of class for a data instance is not correct, it has the least confident value. This strategy considers information only about the most probable label.

Margin Sampling In this strategy, the active learner selects 'k' samples in the pool with the minimum difference between the most probable and second most probable labels.

Entropy Sampling In this strategy, the active learner selects 'k' samples in the pool with the highest entropy. Entropy is the amount of uncertainty or randomness in the outcome of a random variable or an event x.

We knew that higher the entropy value, higher the uncertainty of its label.

Mathematically, Entropy is defined as:

$$\text{Entropy}\ (x) = -\sum_{y \in Y} \mathrm{P}\ (y \mid x) \cdot \log \mathrm{P}\ (y \mid x)$$

where, P $(y \mid x)$ is the posteriori probability that denotes the probability of a random variable 'x' belongs to class 'y'.

The output classes are denoted as $y \in Y = \{y_1, y_2, \ldots, y_k\}$.

© Oxford University Press 2021. All rights reserved

Association Rule Mining

Association rule mining enables us to find patterns in data, occurrences of a specific item, correlations and associations among transactional data items, or causal structures in medical data, etc. Association rule mining has wide applications in the field of business, medical diagnosis, etc. Knowing the buying pattern by rule mining helps to improve sales by providing promotional discount and timely advertisements that attract customers. This is an alternative approach, where classification rules are generated using associative rule mining algorithms. These algorithms search for attribute-value pairs that occur frequently in the data. These frequently occurring items may form association rules, which are then used in classification.

An association rule is generally expressed as:

$$\text{If } (A) \rightarrow \text{Then } (B)$$

© Oxford University Press 2021. All rights reserved

Association Rule Mining

MEASURE 1: SUPPORT

It is measured by the number of transactions that contains the items of both the itemsets, A and B, divided by the total number of transactions. It is measured as the joint probability of A and B.

An association rule is expressed as $\{A \rightarrow B\}$.

Support of $\{A \rightarrow B\}$ = Fraction of transactions that contains both A and B/Total number of transactions.

MEASURE 2: CONFIDENCE

It denotes how likely the itemset B is purchased when itemset A is purchased. In other words, it is measured by the number of transactions with itemset A, in which the itemset B also appears and is expressed as $\{A \rightarrow B\}$.

It is measured as the conditional probability of B given A.

Confidence of $\{A \rightarrow B\}$ = Support {AUB}/Support $\{A\}$

© Oxford University Press 2021. All rights reserved

Association Rule Mining

MEASURE 3: LIFT

Lift is the likelihood of the itemset B being purchased when itemset A is purchased while considering the popularity of B.

$$\text{Lift}\{A \rightarrow B\} = \text{Support}\{\text{AUB}\}/(\text{Support}\{A\} \times \text{Support}\{B\})$$

© Oxford University Press 2021. All rights reserved

Summary

1. Rule-based Learning represents knowledge in the form of IF-THEN rules that proves useful in AI systems.
2. Classification rules can be generated in two ways: (a) Direct method which extracts rules directly from data, (b) Indirect method which extracts rules from other classification models.
3. Rules are generally assessed by two parameters, called Coverage and Accuracy.
4. Generally, rule-based classification models generate more than one rule which may be conflicting and an order is imposed in executing the rules.
5. The most popular strategy is that rules are ordered in the form of a 'decision list' or 'priority list' based on the rule quality measures such as coverage and accuracy or by expert knowledge.
6. OneR algorithm generates a one level decision tree and classifies the test instance based on a single feature/attribute that covers the whole training dataset.
7. Sequential Covering algorithm generates rules without generating a decision tree.
8. Propositional rules are extracted by typical sequential covering algorithms such as PRISM, CN2, RIPPER, etc.
9. Sequential Covering algorithms generate a set of rules sequentially one rule at a time where each rule covers a subset of positive instances from the training dataset.
10. The PRISM algorithm is one such sequential covering algorithm that uses a depth-first search to construct the rules for a given target class C_i.
11. Learning rules in first order form is much more expressive than propositional logic when representing knowledge.

© Oxford University Press 2021. All rights reserved

Summary

12. First order rule learners can express relations between attributes which cannot be expressed by propositional rule learners.
13. Inductive learning of first-order rules is called as Inductive Logic Programming.
14. FOIL (First Order Inductive Learner Algorithm) is a simple inductive learning algorithm to infer rules from data.
15. Explanation Based Learning (EBL) is a deductive learning technique by which an intelligent system can solve a problem by observing and analyzing examples.
16. The principle idea of EBL is to solve a single problem in a particular domain and using the generalized solution, solve similar problems in the future.
17. EBL performs four different learning tasks such as generalization, chunking, operationalization, and analogy.
18. Active learning is a type of interactive semi-supervised learning model which queries unlabelled instances to be labelled by a teacher/human annotator/oracle.
19. Active learning provides greater accuracy with fewer labelled training instances and allows the learner to choose their own training data from which they learn.
20. Association rule mining enables us to find patterns in data, occurrences of a specific item, correlations and associations among transactional data items, or causal structures in medical data, etc.
21. Association rules analysis is a procedure to find how items are associated to each other.
22. Apriori algorithm is a popular association rule mining technique that mines frequent itemsets and association rules in transaction databases.
23. It is called apriori because it uses 'prior' knowledge of frequent itemset properties.
24. Frequent Pattern (FP) Growth algorithm is an improvement over Apriori algorithm where frequent/ significant itemsets are generated without the generation of candidate itemsets.

© Oxford University Press 2021. All rights reserved